Algorithms 2

COMP2711 Discrete Mathematical Tools

Mingxun Zhou (mingxunz@ust.hk)

From a procedure to an analysis

Correctness

Does it return the required answer for every permitted input?

Cost

Which operations are counted, and how many are needed?

From a procedure to an analysis

Correctness

Does it return the required answer for every permitted input?

Cost

Which operations are counted, and how many are needed?

For worst-case analysis, fix the input size and take the **largest** cost.

Lecture roadmap

1. **Sorting: arrange every value.**
   Selection sort $\longrightarrow$ merging $\longrightarrow$ merge sort.

2. **Searching: find only what we need.**
   Linear search $\longrightarrow$ binary search.

3. **Preprocessing: prepare once, answer many queries.**
   Compare repeated scans with sorting once and binary search.

4. **Space cost: use memory to save time.**
   Auxiliary space and a direct-address table.

SECTION 1

Sorting

Sorting problem

Sorting

Rearrange the same values in $A[1..n]$ into **nondecreasing order**. Repeated values are allowed.

Selection sort first, then merge sort.

Selection sort: the human's instinct

Target

Rearrange the values in $A[1..n]$ into **nondecreasing order**.

Interactive visualization

`https://wuwuz.github.io/COMP2711/algorithm-lab/`

Pseudocode

for $i = 1, \ldots, n-1$:
  $nextMinPosition \leftarrow i$; $nextMin \leftarrow A[i]$
  // Find the smallest remaining value.
  for $scan = i+1, \ldots, n$:
    if $A[scan] < nextMin$:
      $nextMin \leftarrow A[scan]$
      $nextMinPosition \leftarrow scan$
  // Put it in position i.
  if $nextMinPosition \neq i$:
    swap $A[i], A[nextMinPosition]$

Selection sort: an example with $n = 5$

Select the smallest remaining value and put it next.

Input	4	2	5	1	3

- Values in their final positions.

Selection sort: an example with $n = 5$

Select the smallest remaining value and put it next.

Input	4	2	5	1	3	
Pass 1	1	2	5	4	3	Select 1; swap with 4.

- Values in their final positions.

Selection sort: an example with $n = 5$

Select the smallest remaining value and put it next.

Input	4	2	5	1	3	
Pass 1	1	2	5	4	3	Select 1; swap with 4.
Pass 2	1	2	5	4	3	Select 2; already in place.

- Values in their final positions.

Selection sort: an example with $n = 5$

Select the smallest remaining value and put it next.

Input	4 2 5 1 3	
Pass 1	1 2 5 4 3	Select 1; swap with 4.
Pass 2	1 2 5 4 3	Select 2; already in place.
Pass 3	1 2 3 4 5	Select 3; swap with 5.

- Values in their final positions.

Selection sort: an example with $n = 5$

Select the smallest remaining value and put it next.

Input	4 2 5 1 3	
Pass 1	1 2 5 4 3	Select 1; swap with 4.
Pass 2	1 2 5 4 3	Select 2; already in place.
Pass 3	1 2 3 4 5	Select 3; swap with 5.
Pass 4	1 2 3 4 5	Select 4; already in place.

- Values in their final positions.

Selection sort: count the work

Count value comparisons.

Comparisons

Pass i scans $n - i$ other values.

$$C(n) = (n-1) + (n-2) + \cdots + 1$$

$$= \frac{n(n-1)}{2} = \Theta(n^2).$$

Selection sort: count the work

Count value comparisons.

Comparisons

Pass i scans $n - i$ other values.

$$C(n) = (n-1) + (n-2) + \cdots + 1$$

$$= \frac{n(n-1)}{2} = \Theta(n^2).$$

Swaps

At most one per pass:

$$\leq n - 1.$$

Even sorted input requires every suffix scan.

Selection sort: count the work

Count value comparisons.

Comparisons

Pass i scans $n - i$ other values.

$$C(n) = (n-1) + (n-2) + \cdots + 1$$

$$= \frac{n(n-1)}{2} = \Theta(n^2).$$

Swaps

At most one per pass:

$$\leq n - 1.$$

Even sorted input requires every suffix scan.

Can a different strategy sort in less than quadratic time?

Divide and conquer

MUST UNDERSTAND **Divide and conquer**

Split into smaller instances of the same problem, solve them, then combine their answers.

1. **Divide:** split the array into two nearly equal halves.
2. **Solve:** sort each half using the same procedure.
3. **Combine:** merge the two sorted halves.

A list of length 0 or 1 is already sorted: the **base case**.

Merging: two sorted lists

Target

Given **sorted** lists L and R, return one sorted list containing all their values.

Interactive visualization

`https://wuwuz.github.io/COMP2711/algorithm-lab/`

Merge(L, R)

$i \leftarrow 1$; $j \leftarrow 1$; *output* $\leftarrow [\,]$
while $i \leq |L|$ **and** $j \leq |R|$:
  // Copy the smaller next value.
  if $L[i] \leq R[j]$:
    append $L[i]$ to *output*; $i \leftarrow i + 1$
  else:
    append $R[j]$ to *output*; $j \leftarrow j + 1$
// One list is exhausted.
append the remaining values to *output*
return *output*

Merge sort: solve the smaller problems

Target

Rearrange the values in $A[1..n]$ into **nondecreasing order**.

Interactive visualization

`https://wuwuz.github.io/COMP2711/algorithm-lab/`

MergeSort(A)

```
// A list of length 0 or 1 is already sorted.
if |A| ≤ 1: return A
split A into nearly equal halves left, right
// Sort each half recursively.
sortedLeft ← MergeSort(left)
sortedRight ← MergeSort(right)
// Combine the sorted halves.
return Merge(sortedLeft, sortedRight)
```

Merge sort: count work by levels

For $n = 2^h$, splitting reaches singletons after $h = \log_2 n$ levels.

Merges at a level	Values per merge	Total work
1	n	$\Theta(n)$
2	$n/2$	$\Theta(n)$
4	$n/4$	$\Theta(n)$
$\vdots$	$\vdots$	$\vdots$

Merge sort: count work by levels

For $n = 2^h$, splitting reaches singletons after $h = \log_2 n$ levels.

Merges at a level	Values per merge	Total work
1	n	$\Theta(n)$
2	$n/2$	$\Theta(n)$
4	$n/4$	$\Theta(n)$
$\vdots$	$\vdots$	$\vdots$

$\Theta(n)$ per level $\times$ $\log_2 n$ merge levels $= \mathbf{\Theta(n \log n)}$.

Merge sort: count work by levels

For $n = 2^h$, splitting reaches singletons after $h = \log_2 n$ levels.

Merges at a level	Values per merge	Total work
1	n	$\Theta(n)$
2	$n/2$	$\Theta(n)$
4	$n/4$	$\Theta(n)$
$\vdots$	$\vdots$	$\vdots$

$\Theta(n)$ per level $\times$ $\log_2 n$ merge levels $= \mathbf{\Theta(n \log n)}$.

General n: balanced halves give the same order (see the note).

SECTION 2

Searching

Searching: find a given value

Problem

Given an array $A[1..n]$ and a target, return an index holding the target, or -1 if it is absent.



$target = 9$

Searching: find a given value

Problem

Given an array $A[1..n]$ and a target, return an index holding the target, or -1 if it is absent.



$target = 9$ answer: index **5**.

For target 6, return -1. Repeated values are allowed.

Linear search

Target

Find a matching index in an **unsorted** array, or return -1.

Interactive visualization

`https://wuwuz.github.io/COMP2711/algorithm-lab/`

LinearSearch(A, *target*)

```
// Check positions from left to right.
for i = 1, ..., n:
    if A[i] = target: return i
// Every position has been ruled out.
return −1
```

Linear search: worst-case cost

Count equality tests between an array value and the target.

Upper bound

Each position is tested at most once.

$$C(n) \leq n.$$

Linear search: worst-case cost

Count equality tests between an array value and the target.

Upper bound

Each position is tested at most once.

$$C(n) \leq n.$$

A worst-case input

An absent target forces all n tests.

$$C_{\text{worst}}(n) = n.$$

Worst-case time: $\boldsymbol{\Theta(n)}$.

Binary search

MUST UNDERSTAND

Binary search

In a **sorted** random-access array, compare with the middle and discard the impossible half.

Interactive visualization

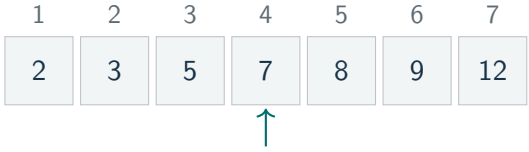
`https://wuwuz.github.io/COMP2711/algorithm-lab/`

BinarySearch$(A, target)$

$low \leftarrow 1$; $high \leftarrow n$
while $low \leq high$:
  $mid \leftarrow low + \lfloor (high - low)/2 \rfloor$
  if $A[mid] = target$: **return** mid
  // Exclude the middle and the impossible half.
  if $A[mid] < target$: $low \leftarrow mid + 1$
  else: $high \leftarrow mid - 1$
return -1

Binary search: keep the possible half

Sorted input; target 9.



low	high	mid	$A[mid]$	Action
1	7	4	7	Too small: keep positions 5..7.

Binary search: keep the possible half

Sorted input; target 9.



low	high	mid	$A[mid]$	Action
1	7	4	7	Too small: keep positions 5..7.
5	7	6	9	Match: return 6.

Binary search: worst-case cost

After each failed probe, at most half the candidates remain.

$$n \longrightarrow \left\lfloor \frac{n}{2} \right\rfloor \longrightarrow \left\lfloor \frac{n}{4} \right\rfloor \longrightarrow \cdots \longrightarrow 0.$$

Worst-case probes, $n \geq 1$

$$\lfloor \log_2 n \rfloor + 1.$$

An absent target above every entry attains this bound.

Binary search: worst-case cost

After each failed probe, at most half the candidates remain.

$$n \longrightarrow \left\lfloor \frac{n}{2} \right\rfloor \longrightarrow \left\lfloor \frac{n}{4} \right\rfloor \longrightarrow \cdots \longrightarrow 0.$$

Worst-case probes, $n \geq 1$

$$\lfloor \log_2 n \rfloor + 1.$$

An absent target above every entry attains this bound.

Each probe uses at most two value comparisons and $O(1)$ other work.

Worst-case time: **$\Theta(\log n)$** $(n \geq 2)$.

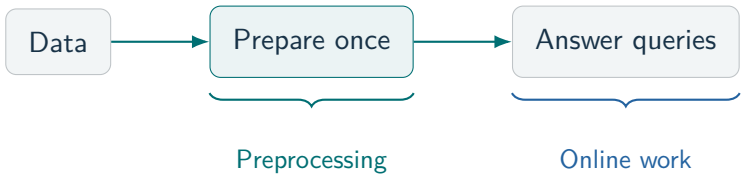
SECTION 3

Preprocessing

Preprocessing

Preprocessing

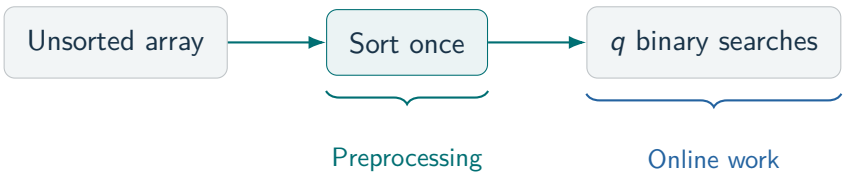
Do some work **once** to make later queries cheaper.



Preprocessing: sorting once

An unsorted array has n numbers. Answer q membership queries.

The array stays **unchanged** between queries.



Preprocessing: sorting once

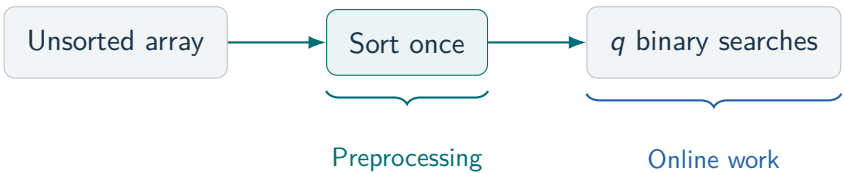
An unsorted array has n numbers. Answer q membership queries.

$$A = [8, 3, 12, 5, 9, 2, 7].$$

Target 9: **Yes**, it occurs in A.

Target 6: **No**, it is absent.

The array stays **unchanged** between queries.



Preprocessing: two search strategies

Same unchanged array: $n \geq 2$ numbers, $q \geq 1$ membership queries.

	Linear search each time	**Sort once + binary search**
Preparation	None	$\Theta(n \log n)$
Per query (worst case)	$\Theta(n)$	$\Theta(\log n)$
Total (worst case)		

Preprocessing: two search strategies

Same unchanged array: $n \geq 2$ numbers, $q \geq 1$ membership queries.

	Linear search each time	**Sort once + binary search**
Preparation	None	$\Theta(n \log n)$
Per query (worst case)	$\Theta(n)$	$\Theta(\log n)$
Total (worst case)	$\Theta(qn)$	$\Theta(n \log n + q \log n)$

Preprocessing: when does sorting pay off?

Compare **worst-case total time**, including the initial sort.

Number of queries	Linear search each time	Sort once + binary search
$q = 1$	$\Theta(n)$	$\Theta(n \log n)$
$q = \lceil \log_2 n \rceil$		
$q = n$		

Preprocessing: when does sorting pay off?

Compare **worst-case total time**, including the initial sort.

Number of queries	Linear search each time	Sort once + binary search
$q = 1$	$\Theta(n)$	$\Theta(n \log n)$
$q = \lceil \log_2 n \rceil$	$\Theta(n \log n)$	$\Theta(n \log n)$
$q = n$		

Preprocessing: when does sorting pay off?

Compare **worst-case total time**, including the initial sort.

Number of queries	Linear search each time	Sort once + binary search
$q = 1$	$\Theta(n)$	$\Theta(n \log n)$
$q = \lceil \log_2 n \rceil$	$\Theta(n \log n)$	$\Theta(n \log n)$
$q = n$	$\Theta(n^2)$	$\Theta(n \log n)$

Many queries share the one-time sorting cost.

SECTION 4

Space cost

Space cost

Auxiliary space

The maximum **extra memory** used at one time, beyond the input.

Count memory cells, just as we count operations for time.

Linear search: input

The array already contains n values.

Linear search: extra memory

An index and a constant number of other variables.

$O(1)$ auxiliary space.

Direct addressing: one slot per possible value

Assume every input value is an integer in $1, \ldots, U$.

Preprocessing

$Present[1..U] \leftarrow$ No
for each x in A:
    $Present[x] \leftarrow$ Yes

Query $x \in [1, U]$

return $Present[x]$

One array access: $O(1)$.

$A = [2, 5, 2, 7], \quad U = 8.$

1	2	3	4	5	6	7	8
No	Yes	No	No	Yes	No	Yes	No

Direct addressing: one slot per possible value

Assume every input value is an integer in $1, \ldots, U$.

Preprocessing

$Present[1..U] \leftarrow$ No
for each x in A:
    $Present[x] \leftarrow$ Yes

Query $x \in [1, U]$

return $Present[x]$

One array access: $O(1)$.

$A = [2, 5, 2, 7], \quad U = 8.$

1	2	3	4	5	6	7	8
No	Yes	No	No	Yes	No	Yes	No

Query 5: **Yes**. Query 6: **No**.

Time and space: compare the costs

Method	Preparation	Per query	Extra space
Linear scan	None	$O(n)$	$O(1)$
Merge sort + binary search	$O(n \log n)$	$O(\log n)$	$O(n)$
Direct-address table	$O(U + n)$	$O(1)$	$O(U)$

It's often about the tradeoff between space and time.

You'll come back to this again and again in the future!

Time and space: compare the costs

Method	Preparation	Per query	Extra space
Linear scan	None	$O(n)$	$O(1)$
Merge sort + binary search	$O(n \log n)$	$O(\log n)$	$O(n)$
Direct-address table	$O(U + n)$	$O(1)$	$O(U)$

It's often about the tradeoff between space and time.

You'll come back to this again and again in the future!

Depending on the application, choose the one that fits.

Summary

Task	Method / input	Worst-case time
Sort all values	Selection sort	$\Theta(n^2)$
	Merge sort	$\Theta(n \log n)$
Find a given value	Linear search	$\Theta(n)$
	Binary search; already sorted	$\Theta(\log n)$
Answer q queries	Sort once, then binary search	$O(n \log n + q \log n)$

Reading

- Rosen 8e: Section 3.1.2 (searching), 3.1.3 (sorting), and 3.3 (cost analysis).
- Merge sort: Section 5.4, an instructor-selected extension.
- Optional reading: median of medians, companion note and CMU 15-451 lecture notes (instructor-selected extension).
- Additional note reading: string matching, duplicate detection, insertion sort, and bubble sort.

Algorithm lab: `https://wuwuz.github.io/COMP2711/algorithm-lab/`

Additional sorting methods

Insertion sort: the procedure

Target

Rearrange the values in $A[1..n]$ into **nondecreasing order**.

Interactive visualization

`https://wuwuz.github.io/COMP2711/algorithm-lab/`

Pseudocode

for $i = 2, \ldots, n$:
  $key \leftarrow A[i]$; $j \leftarrow i - 1$
  // Shift larger values one position right.
  while $j \geq 1$ **and** $A[j] > key$:
    $A[j + 1] \leftarrow A[j]$
    $j \leftarrow j - 1$
  // Insert the saved value into the gap.
  $A[j + 1] \leftarrow key$

Bubble sort: fixed passes

Target

Rearrange the values in $A[1..n]$ into **nondecreasing order**.

Interactive visualization

`https://wuwuz.github.io/COMP2711/algorithm-lab/`

Pseudocode

for $pass = 1, \ldots, n-1$:
  for $j = 1, \ldots, n - pass$:
    // Swap neighbors that are out of order.
    if $A[j] > A[j+1]$:
      swap $A[j], A[j+1]$

Optional: Selection by rank

Finding the median

Selection by rank

Select(A, k) returns the ***k*th smallest value**, counting repeated entries separately. Here $1 \leq k \leq n$.

8	3	12	5	9	2	7

Find the median: rank $\lceil n/2 \rceil$ (the lower middle value if n is even).

Finding the median

Selection by rank

Select(A, k) returns the ***k*th smallest value**, counting repeated entries separately. Here $1 \leq k \leq n$.

8	3	12	5	9	2	7

Find the median: rank $\lceil n/2 \rceil$ (the lower middle value if n is even).

2	3	5	7	8	9	12

Sorting first takes $O(n \log n)$. Can we find just this rank faster?

Selection: partition around a pivot

$$A = [8, 3, 12, 5, 9, 2, 7], \qquad k = 4, \qquad p = 5.$$

Below p: L	Equal to p: E	Above p: G
$[3, 2]$	$[5]$	$[8, 12, 9, 7]$

Selection: partition around a pivot

$$A = [8, 3, 12, 5, 9, 2, 7], \qquad k = 4, \qquad p = 5.$$

Below p: L	**Equal to p: E**	**Above p: G**
$[3, 2]$	$[5]$	$[8, 12, 9, 7]$
Ranks $1, 2$	Rank 3	Ranks $4, 5, 6, 7$

Selection: partition around a pivot

$$A = [8, 3, 12, 5, 9, 2, 7], \qquad k = 4, \qquad p = 5.$$

Below p: L	**Equal to p: E**	**Above p: G**
$[3, 2]$	$[5]$	$[8, 12, 9, 7]$
Ranks $1, 2$	Rank 3	Ranks $4, 5, 6, 7$

Rank 4 becomes rank $4 - 2 - 1 = \mathbf{1}$ in G.

$$\text{Select}(G, 1) = \mathbf{7}.$$

Selection: update the rank

Partition into lists $L < p$, $E = p$, and $G > p$ in one scan.

Where is rank k?	What remains to do?
$k \leq \lvert L \rvert$	Select(L, k)
$\lvert L \rvert < k \leq \lvert L \rvert + \lvert E \rvert$	Return p.
$k > \lvert L \rvert + \lvert E \rvert$	Select$(G, k - \lvert L \rvert - \lvert E \rvert)$

Choosing a useful pivot

An extreme pivot

May discard only one value.

$$n + (n-1) + \cdots = \Theta(n^2).$$

A sufficiently central pivot

Guarantees that a constant fraction can be discarded on either side.

Choose it using **medians of small groups**.

Median of medians: groups of five

Sort each group of five; recursively select the median of their middle values.

1	3	6	8	7
2	4	10	13	11
5	9	12	15	16
14	18	19	17	20
25	24	23	21	22

Medians: $5, 9, 12, 15, 16$.

Pivot $p = 12$

Columns are arranged by their medians only to explain the picture.

Median of medians: groups of five

Sort each group of five; recursively select the median of their middle values.

1	3	6	8	7
2	4	10	13	11
5	9	12	15	16
14	18	19	17	20
25	24	23	21	22

Pivot $p = 12$

Medians: $5, 9, 12, 15, 16$.

Three groups each certify three values ≤ 12.

$$3 \times 3 = 9.$$

Likewise, nine values are ≥ 12.

Columns are arranged by their medians only to explain the picture.

Linear-time selection: the procedure

Target

Find rank k in unsorted A.

Median: $k = \lceil n/2 \rceil$.

Select(A, k)

if $|A| \leq 5$: sort A; **return** $A[k]$
// Sort each full group of five.
$M \leftarrow$ list of their middle values
$p \leftarrow \text{Select}(M, \lceil |M|/2 \rceil)$
// Include leftover values in this scan.
partition A into $L < p$, $E = p$, $G > p$
if $k \leq |L|$: **return** $\text{Select}(L, k)$
if $k \leq |L| + |E|$: **return** p
return $\text{Select}(G, k - |L| - |E|)$

Why a constant fraction is discarded

First suppose n is divisible by 5.

Count values at most p

At least half of the $n/5$ group medians are $\leq p$.

Each such group contributes **three** values $\leq p$.

Why a constant fraction is discarded

First suppose n is divisible by 5.

Count values at most p

At least half of the $n/5$ group medians are $\leq p$.

Each such group contributes **three** values $\leq p$.

$$\frac{1}{2} \cdot \frac{n}{5} \cdot 3 = \frac{3n}{10}.$$

The same count applies to values $\geq p$.

$$|L|, |G| \leq n - \frac{3n}{10} = \frac{\mathbf{7n}}{\mathbf{10}}.$$

For general n: $|L|, |G| \leq 7n/10 + 2$ (rounding proof in the note).

Why the total work is linear

Find the pivot	Find the requested rank
At most $n/5$ values.	At most $7n/10 + 2$ values.

Combined size $\leq 0.9n + 2 \leq 0.95n \quad (n > 40)$.

Why the total work is linear

Find the pivot

At most $n/5$ values.

Find the requested rank

At most $7n/10 + 2$ values.

$$\text{Combined size} \leq 0.9n + 2 \leq 0.95n \quad (n > 40).$$

Each call does linear grouping/partition work. Treat sizes ≤ 40 as small.

$$T(n) \leq c(n + 0.95n + 0.95^2 n + \cdots) = 20cn.$$

Theorem

Median of medians finds any rank in worst-case $O(n)$ time.

Practice: which rank remains?

$$L = [2, 1], \qquad E = [5, 5, 5], \qquad G = [9, 8].$$

Find rank 4

Find rank 6

Practice: which rank remains?

$$L = [2, 1], \qquad E = [5, 5, 5], \qquad G = [9, 8].$$

Find rank 4

Ranks $3, 4, 5$ lie in E.

Return 5.

Find rank 6

Select rank $6 - 2 - 3 = 1$ in G.

Return 8.